DOCUMENT · SOURCE DOCS/IMPLEMENTATION_STATUS.MD

NexusOS implementation status

Evidence-based status of significant modules as of the documentation audit against `main` (application root `ai-business-os/ai-business-os/`).

Status legend

STATUS	MEANING
Implemented	Relevant API + (usually) UI exist; behavior supported by code and/or automated tests
Implemented, partially verified	Code exists; coverage, env gating, or live verification incomplete
Partially implemented	Meaningful gaps in end-to-end capability
Placeholder	Navigation/UI shell without complete backend behavior
Planned	Explicitly deferred / not built
Unverified	Insufficient evidence in this audit pass

This document does **not** invent a single “% complete” number.

01 / PLATFORM FOUNDATION

Platform foundation

FEATURE	STATUS	EVIDENCE	TESTS / CHECKS	LIMITATIONS
Multi-tenant company/user/role/permission model	Implemented	`app/models/`, registration bootstrap	security + auth tests	—
Auth register/login/logout/refresh	Implemented	`app/api/v1/auth.py`, `auth_service.py`	auth + security tests	Email provider env-dependent
Email verify / password reset	Implemented	auth routes + service	frontend + backend auth tests	Delivery needs email provider
JWT + refresh rotation + reuse detection	Implemented	`app/core/security.py`, auth service	security/session tests	—
Argon2 passwords	Implemented	security module + dependency	auth tests	—
RBAC `require_permission`	Implemented	`dependencies/auth.py`	`tests/security/test_authorization.py`	Must be attached per route
Tenant-scoped `BaseRepository`	Implemented	`repositories/base.py` + domain subclasses	`test_tenant_isolation.py`	Not RLS
PostgreSQL RLS	Planned	RLS decision note; no policies in migrations	—	Deferred
Health endpoints	Implemented	`api/v1/health.py`	CI / manual	—
Request context / structured logging	Implemented	middleware + core logging	security logging tests (partial)	—
CORS + security headers	Implemented	`main.py`, middleware	CORS/header tests in CI scope	HSTS default off
Redis rate limiting	Implemented	`rate_limit.py`	security rate-limit tests	**Fails open** if Redis errors
CI pipeline	Implemented	`.github/workflows/ci.yml`	GitHub Actions	Backend suite **scoped**, not full pytest

02 / PRODUCT MODULES

Product modules

MODULE	STATUS	PRIMARY PATHS	TESTS	NOTES
CRM — customers, contacts, leads, deals, notes	Implemented	`api/v1/crm/*`, `repositories/crm.py`, `frontend/src/features/crm`	`test_*_api.py`, FE page tests	Live routers mounted in `api/v1/router.py`
HR — employees, leave, attendance	Implemented	`api/v1/hr/*`, FE HR features	HR API/FE tests	—
Finance — invoices, payments, expenses	Implemented	`api/v1/finance/*`, FE finance	invoice/expense/payment tests	—
Inventory — products, warehouses, stock	Implemented	`api/v1/inventory/*`, FE inventory	inventory-related tests (coverage varies)	—
Projects & tasks	Implemented	`api/v1/projects/*`, FE projects/tasks	projects API tests	—
Reports	Implemented	`api/v1/reports.py`, FE reports	`test_reports_*`, FE tests	Enum/SQL regressions in CI
Dashboard metrics	Implemented	`api/v1/dashboard.py`, FE dashboard	dashboard API/FE tests	Not fixture-only (contrary to old README)
Notifications	Implemented	`api/v1/notifications.py`	related tests	Inbox for current user
Settings	Implemented	`api/v1/settings.py`, FE settings	settings/billing UI	Includes billing/Connect surfaces
AI assistant	Implemented, partially verified	`api/v1/assistant.py`, `services/ai_*`	assistant-related tests if present; needs key	Fail-soft without `OPENAI_API_KEY`
Documents management	Unverified / not first-class	No dedicated router in `api/v1/router.py`	—	Do not claim as shipped module
Design system / shell	Implemented	FE components, routes, layout	FE component/route tests	—

03 / PAYMENTS AND BILLING

Payments and billing

FEATURE	STATUS	EVIDENCE	TESTS	LIMITATIONS
Platform SaaS billing API	Implemented, partially verified	`api/v1/billing.py`, `payments/platform/`	`test_stripe_checkout_api.py`, billing service tests	Requires `STRIPE_BILLING_ENABLED` + secrets/prices
Checkout hosted + Elements	Implemented, partially verified	billing service `ui_mode`	checkout API tests; FE `BillingCheckoutPage` tests	Live Stripe account/env ops-dependent
Stripe Connect onboarding	Implemented, partially verified	`finance/payment_providers.py`, `payments/tenant/connect_service.py`	`test_stripe_connect_onboarding.py`	Connect platform must be enabled in Stripe; Accounts v2 path in code
Stripe webhooks	Implemented, partially verified	`api/v1/webhooks/stripe.py`	`test_stripe_webhooks_api.py`	Signature secrets required
Fail-closed Stripe flags	Implemented	`payments/stripe_config.py` defaults false	unit/service tests	—

Ops guide (may lag dashboard state): `ai-business-os/ai-business-os/docs/ops/Stripe-Payments-and-Billing.md`. Master checklist under `docs/reports/` may still say “BLOCKED” for Stripe keys — treat as **historical** unless re-verified against the live host.

04 / FRONTEND ROUTING NOTE

Frontend routing note

Module screens live under `frontend/src/features/*` and `frontend/src/pages/*` with permission-aware route wrappers (`frontend/src/routes/`). `pages/placeholders/*` still exists for some shell cases; **do not** treat all modules as placeholders — CRM/finance/HR/etc. have real feature pages.

05 / CHECKS EXECUTED DURING DOCUMENTATION AUDIT

Checks executed during documentation audit

CHECK	RESULT	NOTES
Repository / router inventory	Pass	`api/v1/router.py` lists domain modules
Static tenancy/security review	Pass (review)	See MULTI_TENANCY + SECURITY
Prior session: targeted pytest (Stripe + security)	**181 passed**	Not re-run in this docs-only pass
Prior session: `tsc --noEmit`	Pass	Frontend
Prior session: billing vitest	**5 passed**	`src/pages/billing`
Full `pytest -q`	**Not run** this pass	CI also scopes backend tests
`npm run build`	**Not run** this pass	CI runs build on PRs
Railway / Vercel / Cloudflare dashboards	**Not run / unverified**	No CLI retry; no secret access

06 / OUTSTANDING WORK (NON-EXHAUSTIVE, HONESTY-FIRST)

Outstanding work (non-exhaustive, honesty-first)

1. Commit/push root `.gitignore` if not yet on `origin` (local security hygiene).
2. Reconcile or archive outdated claims in older reports (RLS note early sections; master checklist Stripe rows).
3. Expand CI toward broader backend coverage as baseline debt is cleared (workflow comments).
4. Ops verification: trusted proxy CIDRs, HSTS, Stripe live/test mode, Cloudflare controls.
5. Consider RLS only as a dedicated hardening project (not silently implied).

07 / RELATED

Related

- ARCHITECTURE.md
- MULTI_TENANCY.md
- SECURITY.md
- ../README.md